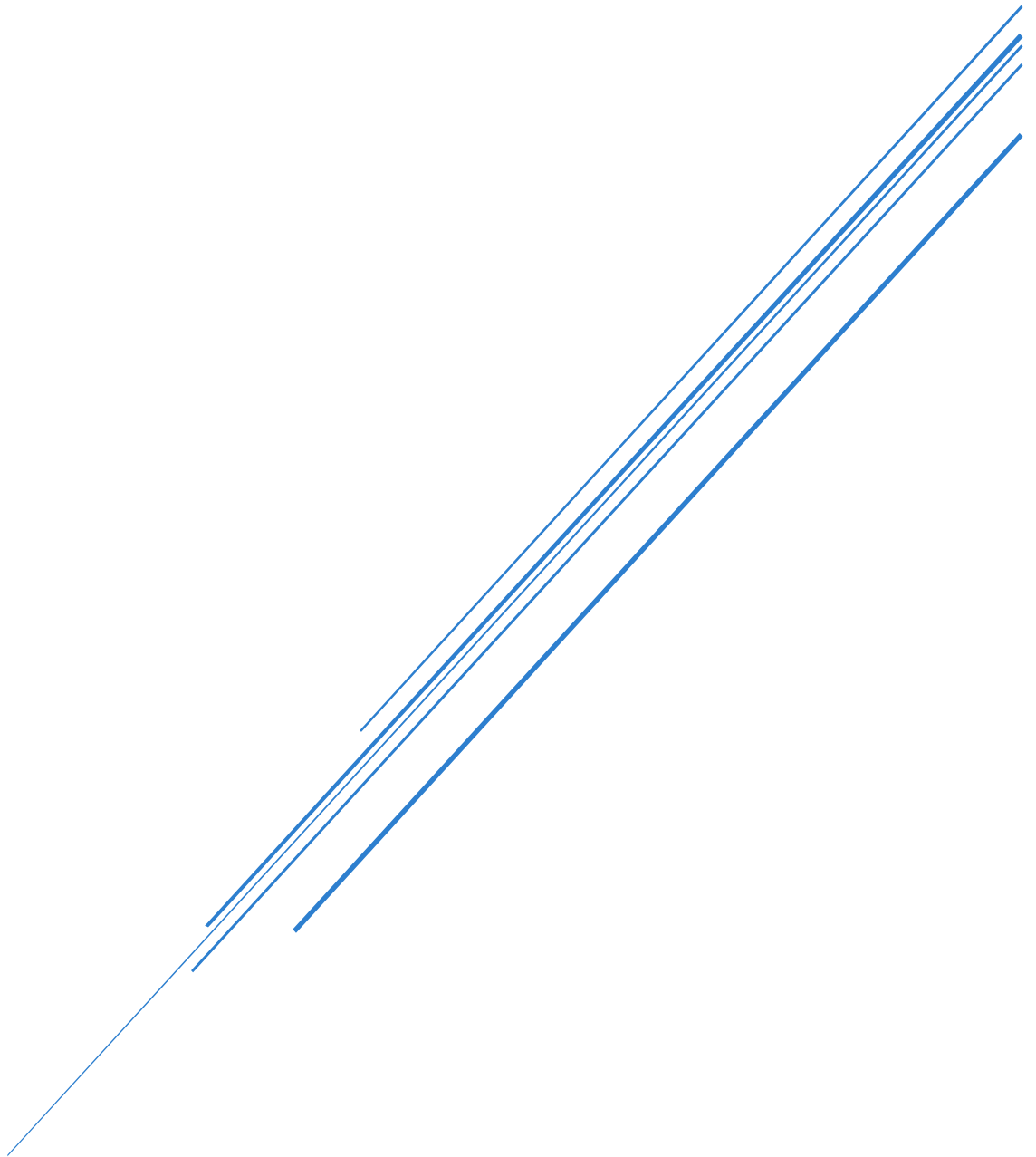


SQL JOINS EXERCISE

NAME: BOTSHELO MOILWENYANE

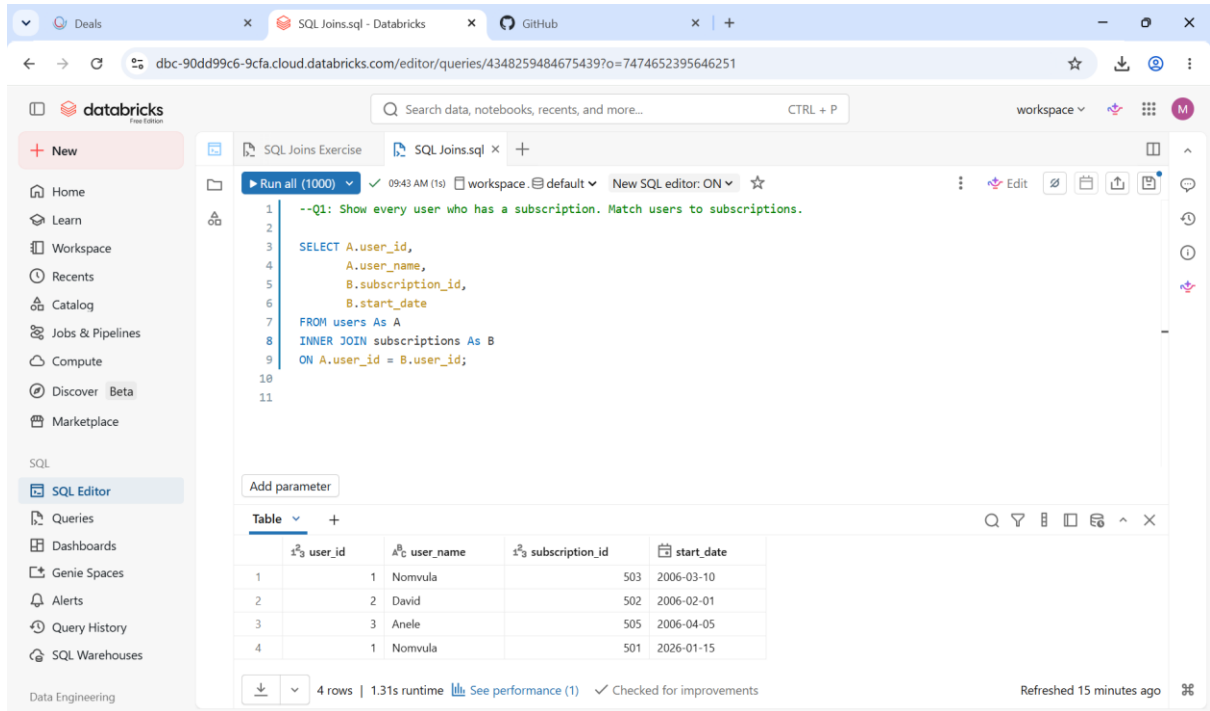
STUDENT NUMBER: 2805001389

COURSE: DATA ANALYTICS WITH SQL



This exercise was completed using the Databricks Spark SQL environment. Tables were created and populated within Databricks, and all queries were written and executed using Spark SQL to retrieve and validate the data. The platform provided an efficient environment for performing the SQL joins exercise.

Q1. Show every user who has a subscription. Match users to subscriptions.



The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
--Q1: Show every user who has a subscription. Match users to subscriptions.

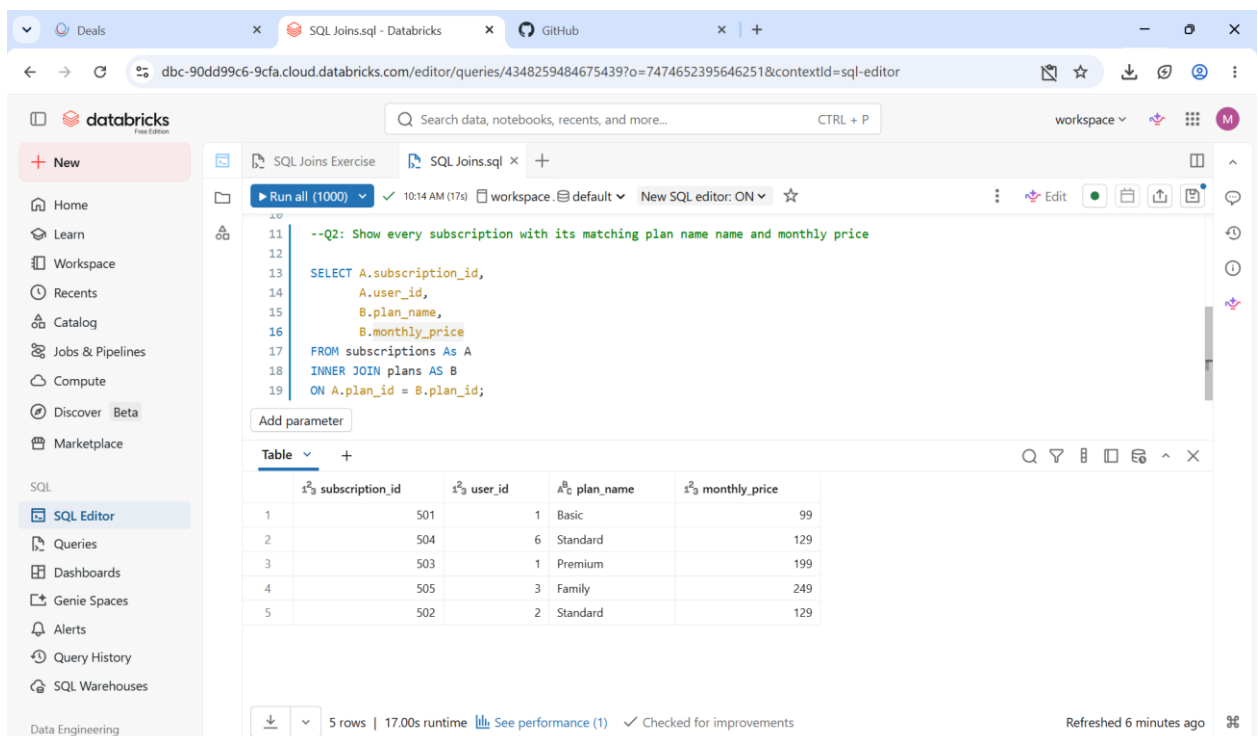
SELECT A.user_id,
       A.user_name,
       B.subscription_id,
       B.start_date
FROM users As A
INNER JOIN subscriptions As B
ON A.user_id = B.user_id;
```

The results table displays 4 rows of data:

	user_id	user_name	subscription_id	start_date
1	1	Nomvula	503	2006-03-10
2	2	David	502	2006-02-01
3	3	Anele	505	2006-04-05
4	1	Nomvula	501	2026-01-15

The interface also shows a sidebar with navigation options and a bottom status bar indicating 4 rows, 1.31s runtime, and a refresh time of 15 minutes ago.

Q2. Show every subscription with its matching plan name and monthly price.



The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
--Q2: Show every subscription with its matching plan name name and monthly price

SELECT A.subscription_id,
       A.user_id,
       B.plan_name,
       B.monthly_price
FROM subscriptions As A
INNER JOIN plans AS B
ON A.plan_id = B.plan_id;
```

The results table displays 5 rows of data:

	subscription_id	user_id	plan_name	monthly_price
1	501	1	Basic	99
2	504	6	Standard	129
3	503	1	Premium	199
4	505	3	Family	249
5	502	2	Standard	129

The interface also shows a sidebar with navigation options and a bottom status bar indicating 5 rows, 17.00s runtime, and a refresh time of 6 minutes ago.

Q3. Show every viewing session that has a matching show. Include the show title and genre.

The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
--Q3 Show every viewing session that has a matching show. Include the show title and genre
SELECT A.session_id,
       B.show_title,
       B.genre,
       A.watch_minutes
FROM viewing_sessions As A
INNER JOIN shows As B
ON A.show_id = B.show_id;
```

The results table displays 4 rows of data:

	session_id	show_title	genre	watch_minutes
1	904	Comedy Hour	Comedy	20
2	903	Crime Time	Drama	60
3	902	Tech Tales	Documentary	30
4	901	Comedy Hour	Comedy	45

At the bottom, it indicates "4 rows | 1.51s runtime" and "Refreshed now".

Q4. Show every viewing session with the user who watched it. Show sessions with a matching user

The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
--Q4 Show every viewing session with the user who watched it. Show sessions with a matching user
SELECT A.user_name,
       A.country,
       B.session_id,
       B.show_id,
       B.watch_minutes
FROM users As A
INNER JOIN viewing_sessions As B
ON A.user_id = B.user_id;
```

The results table displays 4 rows of data:

	user_name	country	session_id	show_id	watch_minutes
1	Nomvula	Johannesburg	903	702	60
2	David	Cape town	902	703	30
3	Anele	Durban	905	705	90
4	Nomvula	Johannesburg	901	701	45

At the bottom, it indicates "4 rows | 1.73s runtime" and "Refreshed now".

Q5. Show users along with their subscriptions, the plan name, and the price. Use only users who have both a subscription and a valid plan.

The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
--Q5 Show users along with their subscriptions, the plan name, and the price. Use only users who have both a subscription and a valid plan.

SELECT A.user_name,
       A.country,
       C.plan_name,
       C.monthly_price,
       B.start_date
FROM users As A
INNER JOIN subscriptions As B
ON A.user_id = B.user_id
INNER JOIN plans As C
ON B.plan_id = C.plan_id;
```

The results table shows 4 rows:

	user_name	country	plan_name	monthly_price	start_date
1	Nomvula	Johannesburg	Premium	199	2006-03-10
2	David	Cape town	Standard	129	2006-02-01
3	Anele	Durban	Family	249	2006-04-05
4	Nomvula	Johannesburg	Basic	99	2026-01-15

4 rows | 0.46s runtime | See performance (1) | Checked for improvements | Refreshed 3 minutes ago

Q6. Show every user and any subscriptions they have. Users without subscriptions must still appear.

The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
--Q6 Show every user and any subscriptions they have. Users without subscriptions must still appear.

SELECT A.user_id,
       A.user_name,
       B.subscription_id,
       B.start_date
FROM users As A
LEFT JOIN subscriptions As B
ON A.user_id = B.user_id;
```

The results table shows 6 rows:

	user_id	user_name	subscription_id	start_date
1	1	Nomvula	503	2006-03-10
2	2	David	502	2006-02-01
3	3	Anele	505	2006-04-05
4	4	Kabelo	null	null
5	5	Lerato	null	null
6	1	Nomvula	501	2026-01-15

6 rows | 14.66s runtime | See performance (1) | Optimize | Refreshed now

Q7. Show every plan and the subscriptions on it. Plans with no subscribers must still appear.

The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
--Q7 Show every plan and the subscriptions on it. Plans with no subscribers must still appear.
SELECT A.plan_id,
       A.plan_name,
       B.subscription_id,
       B.user_id
FROM plans As A
LEFT JOIN subscriptions As B
ON A.plan_id = B.plan_id;
```

The results table displays 6 rows of data:

	plan_id	plan_name	subscription_id	user_id
1	10	Basic	501	1
2	11	Standard	504	6
3	12	Premium	503	1
4	13	Family	505	3
5	14	Mobile	null	null
6	11	Standard	502	2

The interface also shows a sidebar with navigation options and a bottom status bar indicating 6 rows and a 2.06s runtime.

Q8. Show every show and any viewing sessions on it. Shows that were never watched must still appear.

The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
--Q8 Show every show and any viewing sessions on it. Shows that were never watched must still appear.
SELECT A.show_id,
       A.show_title,
       B.session_id,
       B.watch_minutes
FROM shows As A
LEFT JOIN viewing_sessions As B
ON A.show_id = B.show_id;
```

The results table displays 6 rows of data:

	show_id	show_title	session_id	watch_minutes
1	701	Comedy Hour	904	20
2	702	Crime Time	903	60
3	703	Tech Tales	902	30
4	704	Cooking Lab	null	null
5	706	Wild Earth	null	null
6	701	Comedy Hour	901	45

The interface also shows a sidebar with navigation options and a bottom status bar indicating 6 rows and a 3.30s runtime.

Q9. Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear (with NULL user details).

The screenshot shows the Databricks SQL Editor interface. The query for Q9 is as follows:

```
--Q9 Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear (with NULL user details)

SELECT A.session_id,
       A.show_id,
       A.watch_minutes,
       B.user_id,
       B.user_name
FROM viewing_sessions As A
LEFT JOIN users As B
ON A.user_id = B.user_id;
```

The results table shows 5 rows:

	session_id	show_id	watch_minutes	user_id	user_name
1	901	701	45	1	Nomvula
2	902	703	30	2	David
3	903	702	60	1	Nomvula
4	904	701	20	null	null
5	905	705	90	3	Anele

5 rows | 31.17s runtime | See performance (1) | Checked for improvements | Refreshed now

Q10. Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear.

The screenshot shows the Databricks SQL Editor interface. The query for Q10 is as follows:

```
--Q10 Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear.

SELECT A.user_name,
       A.country,
       C.plan_name,
       C.monthly_price
FROM users As A
LEFT JOIN subscriptions As B
ON A.user_id = B.user_id
LEFT JOIN plans As C
ON B.plan_id = C.plan_id;
```

The results table shows 6 rows:

	user_name	country	plan_name	monthly_price
1	Nomvula	Johannesburg	Premium	199
2	David	Cape town	Standard	129
3	Anele	Durban	Family	249
4	Kabelo	Pretoria	null	null
5	Lerato	Port Elizabeth	null	null
6	Nomvula	Johannesburg	Basic	99

6 rows | 2.48s runtime | See performance (1) | Optimize | Refreshed now

Q11. Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist.

The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
--Q11 Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist.

SELECT A.user_id,
       A.user_name,
       B.subscription_id,
       B.start_date
FROM users As A
FULL OUTER JOIN subscriptions As B
ON A.user_id = B.user_id;
```

The results table shows 7 rows of data:

	user_id	user_name	subscription_id	start_date
1	5	Lerato	null	null
2	1	Nomvula	503	2006-03-10
3	3	Anele	505	2006-04-05
4	2	David	502	2006-02-01
5	4	Kabelo	null	null
6	1	Nomvula	501	2026-01-15
7	null	null	504	2006-04-05

The interface also shows a sidebar with navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Alerts, Query History, and SQL Warehouses. The bottom status bar indicates 7 rows, 25.27s runtime, and a refresh time of 1 minute ago.

Q12. Show every plan and every subscription, including plans without subscribers AND any subscription referencing plan that does not exist.

The screenshot shows the Databricks SQL Editor interface. The query editor contains the following SQL code:

```
--Q12 Show every plan and every subscription, including plans without subscribers AND any subscription referencing plan that does not exist.

SELECT A.plan_id,
       A.plan_name,
       B.subscription_id,
       B.user_id
FROM plans As A
FULL OUTER JOIN subscriptions As B
ON A.plan_id = B.plan_id;
```

The results table shows 6 rows of data:

	plan_id	plan_name	subscription_id	user_id
1	11	Standard	504	6
2	14	Mobile	null	null
3	13	Family	505	3
4	12	Premium	503	1
5	10	Basic	501	1
6	11	Standard	502	2

The interface also shows a sidebar with navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Alerts, Query History, and SQL Warehouses. The bottom status bar indicates 6 rows, 1.10s runtime, and a refresh time of now.

Q13. Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist.

The screenshot shows the Databricks SQL Editor interface. The query being executed is:

```
--Q13 Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist.

SELECT A.show_id,
       A.show_title,
       B.session_id,
       B.watch_minutes
FROM shows As A
FULL OUTER JOIN viewing_sessions As B
ON A.show_id = B.show_id;
```

The result table displays the following data:

show_id	show_title	session_id	watch_minutes
704	Cooking Lab	null	null
703	Tech Tales	902	30
701	Comedy Hour	904	20
702	Crime Time	903	60
706	Wild Earth	null	null
701	Comedy Hour	901	45
null	null	905	90

The interface indicates 7 rows and a runtime of 2.18s.

Q14. Show every user and every viewing session, including users with no sessions AND session referencing users who do not exist.

The screenshot shows the Databricks SQL Editor interface. The query being executed is:

```
--Q14 Show every user and every viewing session, including users with no sessions AND session referencing users who do not exist.

SELECT A.user_id,
       A.user_name,
       B.session_id,
       B.show_id,
       B.watch_minutes
FROM users As A
FULL OUTER JOIN viewing_sessions As B
ON A.user_id = B.user_id;
```

The result table displays the following data:

user_id	user_name	session_id	show_id	watch_minutes
5	Lerato	null	null	null
1	Nomvula	903	702	60
3	Anele	905	705	90
2	David	902	703	30
4	Kabelo	null	null	null
1	Nomvula	901	701	45
null	null	904	701	20

The interface indicates 7 rows and a runtime of 1.23s.

Q15. Show every user, every subscription, and every plan in one query-using FULL OUTER JOIN throughout.

The screenshot shows the Databricks SQL Editor interface. The query being executed is a FULL OUTER JOIN of three tables: users, subscriptions, and plans. The query is as follows:

```
151 SELECT A.user_id,  
152        A.user_name,  
153        B.subscription_id,  
154        B.plan_id,  
155        C.plan_name  
156 FROM users As A  
157 FULL OUTER JOIN subscriptions As B  
158 ON A.user_id = B.user_id  
159 FULL OUTER JOIN plans As C  
160 ON B.plan_id = C.plan_id;
```

The results of the query are displayed in a table with 8 rows and 6 columns: user_id, user_name, subscription_id, plan_id, and plan_name. The table shows the following data:

	user_id	user_name	subscription_id	plan_id	plan_name
1	2	David	502	11	Standard
2	null	null	504	11	Standard
3	3	Anele	505	13	Family
4	1	Nomvula	503	12	Premium
5	5	Lerato	null	null	null
6	4	Kabelo	null	null	null
7	1	Nomvula	501	10	Basic
8	null	null	null	null	Mobile

The interface also shows a sidebar with navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, and SQL Editor. The bottom of the interface displays the query execution status: 8 rows, 32.80s runtime, and a refresh button.

Bonus Challenge

Bonus Q1. Which users have not subscribed to any plan?

- User_id 4 Kabelo
- User_id 5 Lerato

Bonus Q2. Which subscriptions reference users that do not exist in the users table?

- Subscription_id 504 User_id 6

Bonus Q3. Which shows have never been watched?

- Show-id 704 Cooking Lab
- Show_id 706 Wild Earth

Bonus Q4. Which viewing sessions reference shows that do not exist?

- Session_id 905

Bonus Q5. Which plans have no subscribers?

- Plan_Id 14 mobile